

A) Historia de UML

El Lenguaje Unificado de Modelado, conocido como **UML (Unified Modeling Language)**, es un lenguaje estándar utilizado en la ingeniería de software para representar, analizar y diseñar sistemas informáticos. UML permite describir de manera visual la estructura y el comportamiento de un sistema mediante diagramas, facilitando la comprensión del funcionamiento del software antes de que sea desarrollado. Gracias a esta herramienta, los equipos de desarrollo pueden planificar mejor sus proyectos, identificar posibles problemas y organizar de forma clara los componentes del sistema.

La historia de UML se remonta a la década de **1990**, cuando el desarrollo de software comenzó a experimentar un crecimiento significativo. Durante ese período surgieron diversos métodos de modelado orientados a objetos, cada uno con su propia forma de representar los sistemas. Esta diversidad generaba dificultades para los desarrolladores, ya que cada metodología utilizaba símbolos, diagramas y reglas diferentes, lo que complicaba la comunicación entre equipos de trabajo y organizaciones.

Ante esta situación, surgió la necesidad de crear un lenguaje de modelado que unificara los distintos métodos existentes y permitiera establecer un estándar común en la industria del software. Esta iniciativa fue impulsada por tres importantes expertos en ingeniería de software: **Grady Booch, James Rumbaugh e Ivar Jacobson**, quienes posteriormente serían conocidos como *“Los Tres Amigos”* debido a su colaboración en la creación de UML.

Cada uno de estos especialistas había desarrollado previamente su propio método de modelado orientado a objetos. Grady Booch había creado el **método Booch**, que se enfocaba principalmente en el diseño detallado de sistemas complejos. James Rumbaugh desarrolló la **Técnica de Modelado de Objetos (OMT)**, la cual estaba orientada al análisis estructural de los sistemas. Por su parte, Ivar Jacobson desarrolló el método **OOSE (Object-Oriented Software Engineering)**, que introdujo el concepto de **casos de uso**, una técnica utilizada para describir la interacción entre los usuarios y el sistema.

En el año **1995**, Booch y Rumbaugh comenzaron a trabajar juntos en la empresa **Rational Software** para combinar sus metodologías en un solo lenguaje de modelado.

Posteriormente, Jacobson se unió al proyecto, contribuyendo con su enfoque basado en casos de uso. Como resultado de esta colaboración surgió UML, un lenguaje que integraba las mejores características de los distintos métodos de modelado existentes.

En **1997**, UML fue adoptado oficialmente por el **Object Management Group (OMG)**, una organización internacional encargada de establecer estándares tecnológicos para la industria del software. A partir de ese momento, UML se convirtió en un estándar ampliamente utilizado para el análisis y diseño de sistemas orientados a objetos.

Con el paso del tiempo, UML ha evolucionado mediante diferentes versiones que han ampliado sus capacidades y mejoraron su estructura. Actualmente, UML permite representar distintos aspectos de un sistema mediante diversos tipos de diagramas, como diagramas de clases, diagramas de casos de uso, diagramas de secuencia, diagramas de actividades y diagramas de componentes. Estos diagramas facilitan la comprensión del sistema y permiten que los desarrolladores visualicen su funcionamiento antes de comenzar la programación.

En la actualidad, UML se utiliza ampliamente en el desarrollo de software, ya que permite mejorar la planificación de proyectos, facilitar la comunicación entre los miembros del equipo de desarrollo y reducir errores durante las etapas de diseño. Por estas razones, UML se ha convertido en una herramienta esencial dentro del análisis y diseño de sistemas informáticos.



B) Requerimientos de Sistemas, Funcionales y No Funcionales

En el desarrollo de sistemas informáticos, los **requerimientos** representan uno de los elementos más importantes dentro del proceso de creación de software. Estos describen las necesidades, condiciones y características que un sistema debe cumplir para satisfacer los objetivos de los usuarios o de una organización. Los requerimientos permiten definir con claridad qué debe hacer el sistema, cómo debe funcionar y bajo qué condiciones debe operar.

Antes de iniciar el diseño o la programación de un sistema, es fundamental realizar un análisis detallado de los requerimientos. Este proceso permite comprender las necesidades del usuario y establecer las funciones que el software deberá cumplir. Una correcta definición de los requerimientos ayuda a evitar errores durante el desarrollo, facilita la planificación del proyecto y permite que el sistema final cumpla con las expectativas establecidas.

Dentro de la ingeniería de software, los requerimientos suelen clasificarse en diferentes categorías. Entre las más importantes se encuentran los **requerimientos de sistema, los requerimientos funcionales y los requerimientos no funcionales**. Cada uno de estos cumple un papel específico dentro del desarrollo del sistema.

Requerimientos de Sistema

Los **requerimientos de sistema** describen las características generales y las condiciones necesarias para que el sistema funcione correctamente dentro de un entorno tecnológico determinado. Estos requerimientos establecen el contexto en el cual el software será desarrollado e implementado.

Entre los aspectos que pueden incluirse dentro de los requerimientos de sistema se encuentran el tipo de hardware necesario, el sistema operativo compatible, los recursos tecnológicos requeridos, la infraestructura de red y la interacción con otros sistemas informáticos. Estos elementos permiten definir el entorno técnico donde el sistema deberá operar.

Por ejemplo, un sistema de gestión de ventas puede requerir una computadora con un sistema operativo específico, una base de datos para almacenar información y una conexión a internet para sincronizar datos o realizar consultas externas. Estas condiciones forman parte de los requerimientos de sistema porque determinan los recursos necesarios para que el software funcione correctamente.

Requerimientos Funcionales

Los **requerimientos funcionales** describen las funciones o actividades específicas que el sistema debe realizar. Estos requerimientos se enfocan en el comportamiento del sistema y en las acciones que debe ejecutar para cumplir con las necesidades del usuario.

En otras palabras, los requerimientos funcionales responden a la pregunta **“qué debe hacer el sistema”**. Estos incluyen las operaciones, procesos y servicios que el sistema debe proporcionar a los usuarios.

Por ejemplo, en un sistema de control de ventas algunos requerimientos funcionales pueden ser registrar productos, almacenar información de clientes, procesar ventas, generar facturas, actualizar el inventario y producir reportes de ventas. Cada una de estas funciones representa una actividad que el sistema debe ser capaz de realizar.

Los requerimientos funcionales también pueden incluir reglas de negocio que el sistema debe respetar. Por ejemplo, validar los datos ingresados por el usuario, calcular automáticamente el total de una compra o impedir la venta de productos que no estén disponibles en el inventario.



Requerimientos No Funcionales

Los **requerimientos no funcionales** se refieren a las características de calidad que debe tener el sistema y a las condiciones bajo las cuales debe operar. A diferencia de los requerimientos funcionales, estos no describen funciones específicas, sino la manera en que el sistema debe comportarse mientras realiza sus tareas.

Estos requerimientos están relacionados con aspectos como el **rendimiento, la seguridad, la confiabilidad, la disponibilidad, la usabilidad y la eficiencia del sistema**. Su objetivo es garantizar que el software funcione de manera adecuada y proporcione una buena experiencia a los usuarios.

Por ejemplo, un requerimiento no funcional puede establecer que el sistema debe responder a las consultas en un tiempo determinado, que debe proteger la información mediante mecanismos de seguridad o que debe ser fácil de utilizar incluso para usuarios con poca experiencia en informática.

Otros ejemplos de requerimientos no funcionales incluyen la capacidad del sistema para soportar múltiples usuarios al mismo tiempo, la disponibilidad del sistema durante ciertos horarios o la compatibilidad con diferentes dispositivos y navegadores.

En conclusión, los requerimientos de sistema, funcionales y no funcionales son fundamentales dentro del desarrollo de software, ya que permiten establecer las bases sobre las cuales se construirá el sistema. Su correcta identificación y documentación facilita el trabajo del equipo de desarrollo y garantiza que el software cumpla con las necesidades del usuario de manera eficiente y confiable.

